Practical – 01

Name: Anuj Dhanke

Roll no.: 20

Batch: A1

Title: Write a program non-recursive and recursive program to calculate Fibonacci numbers and analyze their time and space complexity

Input:

1. Non-Recursive (Iterative) Program:

```
def fibonacci_iterative(n):
    a, b = 0, 1

    for i in range(n):
        print(a, end=" ")
        a, b = b, a + b


n = int(input("Enter the number of terms: "))

print("Fibonacci Series:")
fibonacci_iterative(n)
```

2. Recursive Program:

```
3. def fibonacci_recursive(n):
       if n <= 1:
           return n

       return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)

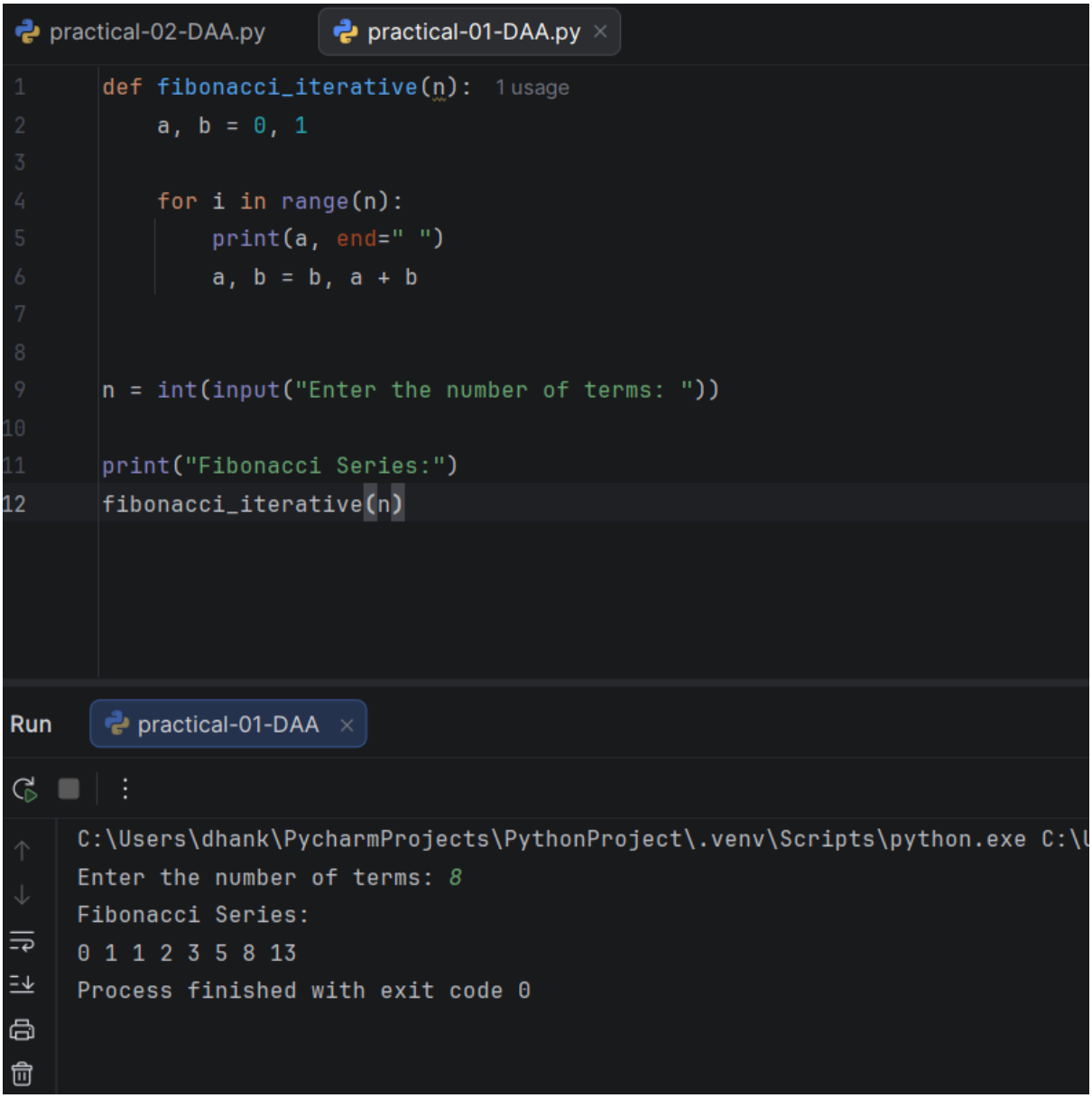

   n = int(input("Enter the number of terms: "))

   print("Fibonacci Series:")

   for i in range(n):
       print(fibonacci_recursive(i), end=" ")
```

Output:

1. Non-Recursive (Iterative) Program:



- **Time Complexity:** O(n) - The loop executes n times.
- **Space Complexity:** O(1) - Only a few variables (a, b, i) are used.

2. Recursive Program:

practical-02-DAA.py practical-01-DAA.py

```python
def fibonacci_recursive(n):  3 usages
    if n <= 1:
        return n

    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)


n = int(input("Enter the number of terms: "))

print("Fibonacci Series:")

for i in range(n):
    print(fibonacci_recursive(i), end=" ")
```

Run practical-01-DAA

```
C:\Users\dhank\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\
Enter the number of terms: 8
Fibonacci Series:
0 1 1 2 3 5 8 13
Process finished with exit code 0
```

- **Time Complexity:** O(2^n)
  More precisely, it is O(ϕ^n), where ϕ ≈ 1.618, but O(2^n) is commonly used in basic analysis.
- **Space Complexity:** O(n)
  - Due to the maximum depth of the recursive call stack.